EVALUACIÓN DE PYTHON – NIVEL JUNIOR
SOLUCIONARIO (USO DEL INSTRUCTOR)

Listas, Tuplas, Diccionarios y Funciones

Fecha: Miércoles 14 de octubre de 2026	**Horario:** 4:00 pm – 6:00 pm
Duración máxima: 2 horas	**Puntaje total:** 100 puntos (+5 bonus)

Criterio general para código (Partes III y IV): **60%** funciona correctamente, **20%** usa la estructura de datos adecuada, **20%** claridad (nombres, orden, sin repetir código). Cualquier solución correcta distinta a la mostrada es válida.

CASO: INVENTARIO DE "TECNOMARKET"

Datos para todo el examen

TecnoMarket guarda sus productos en una **lista de tuplas**. Con ella se construirá un **diccionario** de inventario y se programarán **funciones** para consultarlo y actualizarlo.

```
productos = [
    ("Mouse", 25000, 10),
    ("Teclado", 60000, 3),
    ("Monitor", 450000, 5),
    ("Audifonos", 80000, 2),
    ("Cable HDMI", 15000, 20),
]
# Cada tupla: (nombre, precio, stock)
```

PARTE I – Conceptos

20 puntos · 4 pts c/u · ~15 min

1. ¿Cuál es la diferencia principal entre una lista y una tupla? Da un ejemplo del caso TecnoMarket donde convenga usar tupla.

Respuesta: Lista: mutable (se puede agregar, quitar y modificar elementos). Tupla: inmutable. Conviene tupla para un registro fijo como `("Mouse", 25000, 10)`, o para devolver varios valores de una función. (2 pts diferencia + 2 pts ejemplo coherente).

2. Dado `t = ("Mouse", 25000, 10)`, ¿qué ocurre al ejecutar `t[1] = 30000`? ¿Y si `t` fuera una lista?

Respuesta: Con la tupla se produce `TypeError` (no admite asignación). Con una lista sí se modifica y `t[1]` pasa a valer 30000. (2 pts cada caso).

3. Dado `inv = {"Mouse": 10}`, explica la diferencia entre `inv["Teclado"]` e `inv.get("Teclado", 0)`.

Respuesta: `inv["Teclado"]` lanza `KeyError` si la clave no existe; `inv.get("Teclado", 0)` no falla y devuelve el valor por defecto 0. (2 pts cada comportamiento).

4. ¿Cuál es la diferencia entre `print` y `return` dentro de una función? ¿Qué devuelve una función que no tiene `return`?

Respuesta: `print` solo muestra en pantalla; `return` entrega un valor que se puede guardar o usar. Sin `return` devuelve `None`. (2 pts diferencia + 2 pts None).

5. ¿Qué es un parámetro con valor por defecto? Escribe la firma de una función `saludar` que reciba `nombre` y un `saludo` que por defecto sea "Hola".

Respuesta: Es un parámetro que toma un valor si no se envía argumento. Ejemplo: `def saludar(nombre, saludo="Hola"):` (2 pts definición + 2 pts firma correcta).

PARTE II – Predice la salida

15 puntos · 5 pts c/u · ~15 min

Escribe exactamente lo que imprimirá cada bloque (sin ejecutarlo).

1.

```
numeros = [4, 8, 15, 16, 23]
numeros.append(42)
numeros.remove(8)
print(numeros[1:4])
print(len(numeros))
```

```
Salida:
[15, 16, 23]
5
```

2.

```
stock = {"Mouse": 10, "Teclado": 3}
stock["Mouse"] += 5
stock["Monitor"] = 2
del stock["Teclado"]
print(stock)
print("Teclado" in stock)
```

```
Salida:
{'Mouse': 15, 'Monitor': 2}
False
```

3.

```
def min_max(lista):
    return min(lista), max(lista)

resultado = min_max([7, 2, 9, 4])
print(resultado)
print(type(resultado))
print(resultado[1])
```

```
Salida:
(2, 9)
<class 'tuple'>
9
```

Puntaje: 5 pts por bloque completo; si hay un error en una sola línea de salida, otorgar 3 pts. En el bloque 3, la clave es reconocer que una función que devuelve dos valores entrega una **tupla**.

PARTE III – Funciones sobre el inventario

45 puntos · ~55 min

Escribe cada función. Todas reciben datos por parámetro (no uses variables globales dentro de las funciones).

1. crear_inventario(productos) – 10 pts

Recibe la lista de tuplas `productos` y devuelve un **diccionario** con la forma:

```
{"Mouse": {"precio": 25000, "stock": 10},
 "Teclado": {"precio": 60000, "stock": 3}, ...}
```

```
def crear_inventario(productos):
    inventario = {}
    for nombre, precio, stock in productos:
        inventario[nombre] = {"precio": precio, "stock": stock}
    return inventario
```

Criterios: Recorre la lista desempaquetando cada tupla (`nombre, precio, stock`) y arma el diccionario. 4 pts recorrer/desempaquetar, 4 pts estructura correcta del diccionario, 2 pts `return`.

2. valor_total(inventario) – 8 pts

Devuelve el valor total del inventario: la suma de `precio × stock` de todos los productos.

```
Con los datos de TecnoMarket el resultado es: 3140000
```

```
def valor_total(inventario):
    total = 0
    for datos in inventario.values():
        total += datos["precio"] * datos["stock"]
    return total
```

Criterios: 4 pts recorrer `.values()`/`.items()`, 3 pts acumular correctamente, 1 pt `return`.

3. productos_bajo_stock(inventario, minimo=5) – 8 pts

Devuelve una **lista** con los nombres de los productos cuyo stock sea **menor** que `minimo`, ordenada alfabéticamente.

```
Con los datos de TecnoMarket: ['Audifonos', 'Teclado']
```

```
def productos_bajo_stock(inventario, minimo=5):
    bajos = []
    for nombre, datos in inventario.items():
        if datos["stock"] < minimo:
            bajos.append(nombre)
    bajos.sort()
    return bajos
```

Criterios: 3 pts condición correcta (**<**, no **<=**), 2 pts parámetro por defecto, 3 pts lista ordenada (`sort`/`sorted`).

4. producto_mas_caro(inventario) – 9 pts

Devuelve una **tupla** (`nombre, precio`) del producto de mayor precio.

```
Con los datos de TecnoMarket: ('Monitor', 450000)
```

```
def producto_mas_caro(inventario):
    mejor = None
    for nombre, datos in inventario.items():
        if mejor is None or datos["precio"] > mejor[1]:
            mejor = (nombre, datos["precio"])
    return mejor
```

Criterios: 4 pts lógica del máximo, 3 pts devolver **tupla** de dos valores, 2 pts manejo del caso inicial (`None` o primer elemento).

5. aplicar_descuento(inventario, porcentaje) – 10 pts

Devuelve un **nuevo** inventario con todos los precios rebajados en `porcentaje`% (el stock no cambia). **El inventario original no debe modificarse.**

```
aplicar_descuento(inv, 10)["Mouse"]  ->  {'precio': 22500.0, 'stock': 10}
inv["Mouse"]                         ->  {'precio': 25000, 'stock': 10}
```

```
def aplicar_descuento(inventario, porcentaje):
    nuevo = {}
    for nombre, datos in inventario.items():
        nuevo[nombre] = {
            "precio": datos["precio"] * (1 - porcentaje / 100),
            "stock": datos["stock"],
        }
    return nuevo
```

Criterios: 4 pts cálculo correcto del precio, 4 pts original intacto (crear diccionarios nuevos, no asignar sobre el mismo), 2 pts stock conservado.

PARTE IV – Ejercicio integrador

20 puntos · ~30 min

TecnoMarket registra las ventas del día como una **lista de tuplas** `(producto, cantidad)`:

```
ventas = [("Mouse", 3), ("Monitor", 6), ("Teclado", 1), ("Parlante", 2), ("Mouse", 4)]
```

6. procesar_ventas(inventario, ventas) – 14 pts

Recorre las ventas y **descuenta del stock** del inventario las unidades vendidas. Una venta se **rechaza** si el producto no existe o si la cantidad supera el stock disponible en ese momento (las ventas se procesan en orden). La función devuelve una **tupla** `(ingresos, rechazadas)`, donde `ingresos` es el dinero total de las ventas aceptadas y `rechazadas` es una lista de tuplas `(producto, cantidad, motivo)` con motivo "No existe" o "Stock insuficiente".

```
ingresos, rechazadas = procesar_ventas(inv, ventas)
print(ingresos)     # 235000
print(rechazadas)   # [('Monitor', 6, 'Stock insuficiente'), ('Parlante', 2, 'No existe')]
print(inv["Mouse"]["stock"])   # 3
```

```
def procesar_ventas(inventario, ventas):
    ingresos = 0
    rechazadas = []
    for producto, cantidad in ventas:
        if producto not in inventario:
            rechazadas.append((producto, cantidad, "No existe"))
        elif cantidad > inventario[producto]["stock"]:
            rechazadas.append((producto, cantidad, "Stock insuficiente"))
        else:
            inventario[producto]["stock"] -= cantidad
            ingresos += cantidad * inventario[producto]["precio"]
    return ingresos, rechazadas
```

Criterios: 3 pts desempaquetar cada venta, 3 pts validar que exista el producto (`in`), 3 pts validar stock, 2 pts actualizar stock y acumular ingresos, 3 pts devolver la tupla correcta. Error frecuente: validar contra el stock *inicial* en lugar del actual (el Mouse: 3 + 4 = 7 vendidas con stock 10, ambas válidas).

7. imprimir_reporte(inventario, ingresos, rechazadas) – 6 pts

Imprime el reporte con exactamente este formato (el stock final va ordenado por nombre):

```
=== REPORTE DE VENTAS ===
Ingresos totales: $235,000
Ventas rechazadas:
  - Monitor x6: Stock insuficiente
  - Parlante x2: No existe
Stock final:
  Audifonos: 2
  Cable HDMI: 20
  Monitor: 5
  Mouse: 3
  Teclado: 2
```

```
def imprimir_reporte(inventario, ingresos, rechazadas):
    print("=== REPORTE DE VENTAS ===")
    print(f"Ingresos totales: ${ingresos:,.0f}")
    print("Ventas rechazadas:")
    for producto, cantidad, motivo in rechazadas:
        print(f"  - {producto} x{cantidad}: {motivo}")
    print("Stock final:")
    for nombre in sorted(inventario):
        print(f"  {nombre}: {inventario[nombre]['stock']}")
```

Criterios: 2 pts encabezado e ingresos con formato, 2 pts recorrer rechazadas, 2 pts stock final ordenado. Aquí sí se usa `print` (la función no devuelve nada).

BONUS (+5 pts, solo si terminaste todo): clasificar_por_precio(inventario)

Devuelve un diccionario de **listas** con las claves `"economicos"` (precio menor a 50.000), `"medios"` (entre 50.000 y 200.000, inclusive) y `"premium"` (mayor a 200.000), con los nombres de los productos de cada grupo.

```
{'economicos': ['Mouse', 'Cable HDMI'],
 'medios': ['Teclado', 'Audifonos'],
 'premium': ['Monitor']}
```

```
def clasificar_por_precio(inventario):
    rangos = {"economicos": [], "medios": [], "premium": []}
    for nombre, datos in inventario.items():
        p = datos["precio"]
        if p < 50000:
            rangos["economicos"].append(nombre)
        elif p <= 200000:
            rangos["medios"].append(nombre)
        else:
            rangos["premium"].append(nombre)
    return rangos
```